

Estrutura de Dados II

ÁRVORE B

Prof. CAMILLA LOBO PAULINO

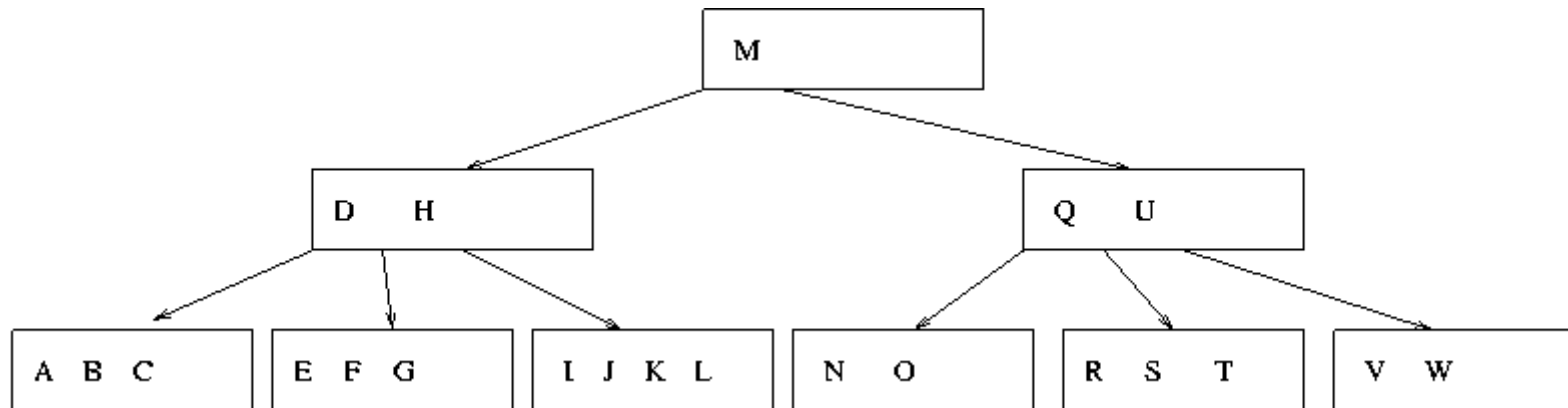
Camilla.paulino@uva.br

- Criada por Bayer e McCreight em 1972 quando trabalhavam no Boeing Scientific Research Labs.
- "B" não significa Binária!
- A origem do "B" é desconhecida. Hipóteses:
 - ✓ Boeing Scientific Research Labs.
 - ✓ Bayer trees (o nome de um dos criadores).
 - ✓ Balanced trees (árvores balanceadas).

- Uma **Árvore B** é um tipo de Árvore de Busca:
 - ✓ Seus nós podem ter N filhos (árvore de ordem N);
 - ✓ É uma árvore balanceada;
 - ✓ Voltada para o armazenamento de grande volume de dados com rápido acesso a um nó específico.

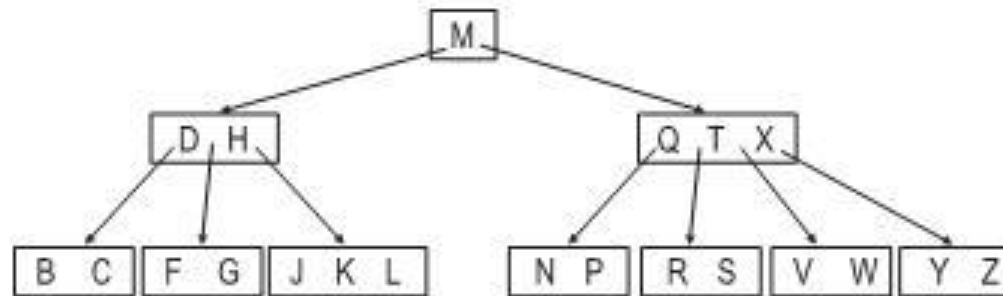
- Uma **Árvore B** de ordem **N** ($N > 2$) possui as seguintes propriedades:
1. Nó raiz:
 - ✓ É um nó folha ou possui, no mínimo, 2 descendentes.
 2. Nós folha:
 - ✓ Possuem de $\lceil N/2 \rceil - 1$ a $N - 1$ chaves.
 - ✓ Estão todos no mesmo nível.
 3. Nós internos:
 - ✓ Possuem de $\lceil N/2 \rceil$ a N descendentes.
 - ✓ Possuem de $\lceil N/2 \rceil - 1$ a $N - 1$ chaves (número de chaves = descendentes $- 1$).
 4. Dentro de um nó, as chaves estão ordenadas.
 5. Dado um nó qualquer e uma chave qualquer desse nó:
 - ✓ Descendentes à esquerda da chave $<$ chave $<$ descendentes à direita da chave.

- Exemplo: árvore B de ordem 5.



1. Nó raiz:
 - ✓ É um nó folha ou possui, no mínimo, 2 descendentes.
2. Nós folha:
 - ✓ Possuem de 2 a 4 chaves.
 - ✓ Estão todos no mesmo nível.
3. Nós internos:
 - ✓ Possuem de 3 a 5 descendentes.
 - ✓ Possuem de 2 a 4 chaves (número de chaves = descendentes – 1).

- Exemplo: árvore B de ordem 4.

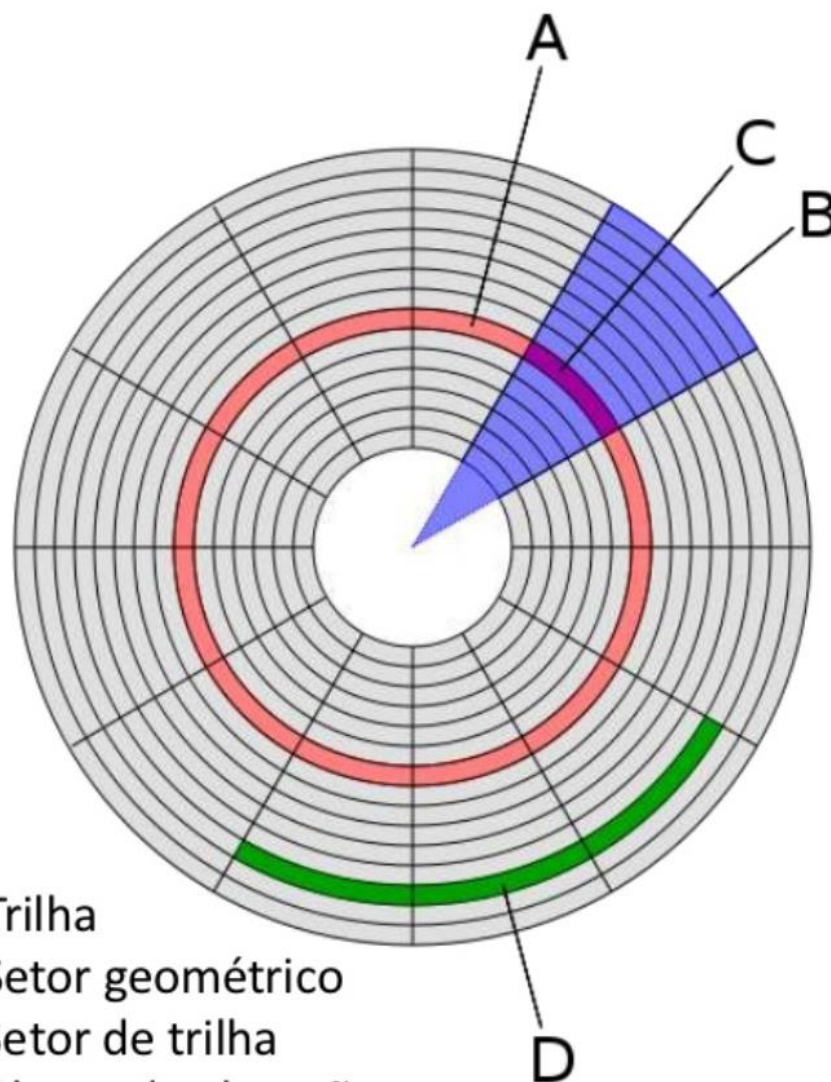


1. Nó raiz:
 - ✓ É um nó folha ou possui, no mínimo, 2 descendentes.
2. Nós folha:
 - ✓ Possuem de 1 a 3 chaves.
 - ✓ Estão todos no mesmo nível.
3. Nós internos:
 - ✓ Possuem de 2 a 4 descendentes.
 - ✓ Possuem de 1 a 3 chaves (número de chaves = descendentes – 1).

- Árvores B foram projetadas para:
 - ✓ O armazenamento de **grandes volume de dados** com rápido acesso a um nó específico;
 - ✓ Funcionar bem em **discos magnéticos** ou outros dispositivos de armazenamento secundário.
- Os algoritmos de tratamento da Árvore B preveem que somente alguns nós da árvore estarão em memória.
- Para evitar um número excessivo de read/write no disco, os nós da árvore são, geralmente, do mesmo tamanho do bloco de alocação do disco.
- Muitos SGBDs usam árvores B ou variações de árvores B para armazenar dados e/ou índices.

Discos

- Tecnologia barata;
- Alta capacidade armazenamento;
- Informações armazenadas em trilhas;
- Trilhas são divididas em setores;
- **Aplicações sempre acessam o disco em unidades de blocos:**
 - Exemplo: 1kb a 8kb;
 - Acesso ao disco é muito custoso.



- (A) Trilha
- (B) Setor geométrico
- (C) Setor de trilha
- (D) Blocos de alocação

- Assumindo que o nó raiz está permanentemente na memória, serão necessários somente 4 acessos ao disco para buscar qualquer chave!



10.100 chaves no nível 1

1.020.100 chaves no nível 2

103.030.100 chaves no nível 3

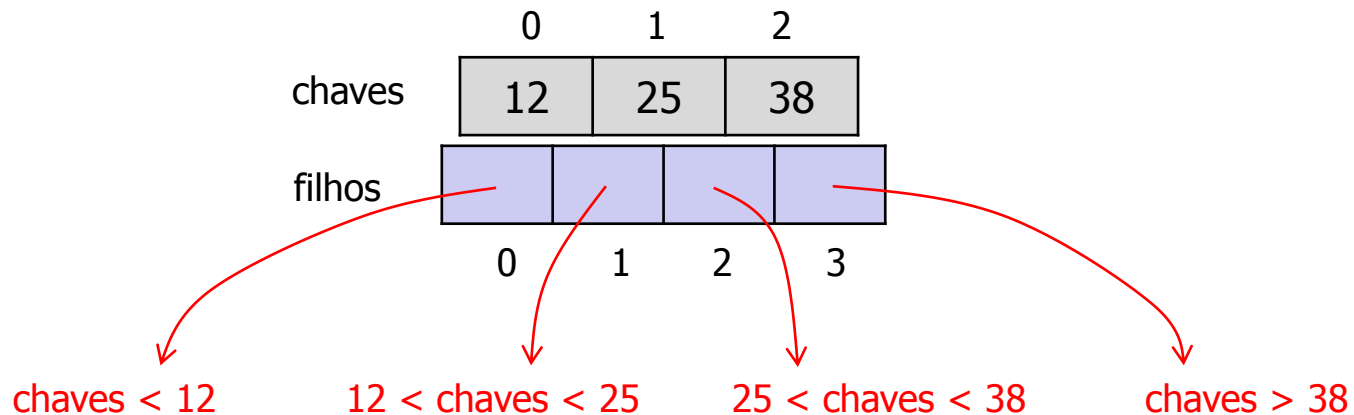
10.406.040.100 chaves no nível 4

- A estrutura do nó de uma Árvore B é mais complexa do que o nó de uma Árvore Binária:

```
const int ORDEM = 5;  // número máximo de filhos de um nó

struct no_btree {
    int num_chaves;           // número de chaves do nó
    bool folha;              // indicador de nó folha
    int chaves[ORDEM-1];      // vetor de chaves
    struct no_btree *filhos[ORDEM]; // vetor de ponteiros
};
```

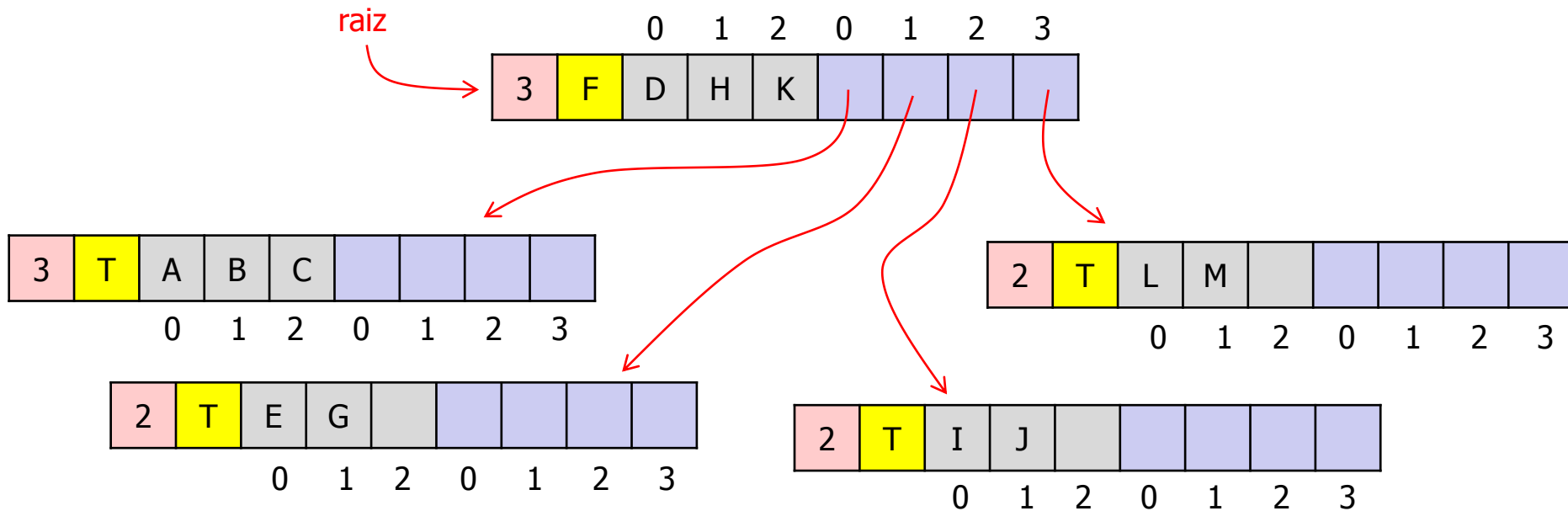
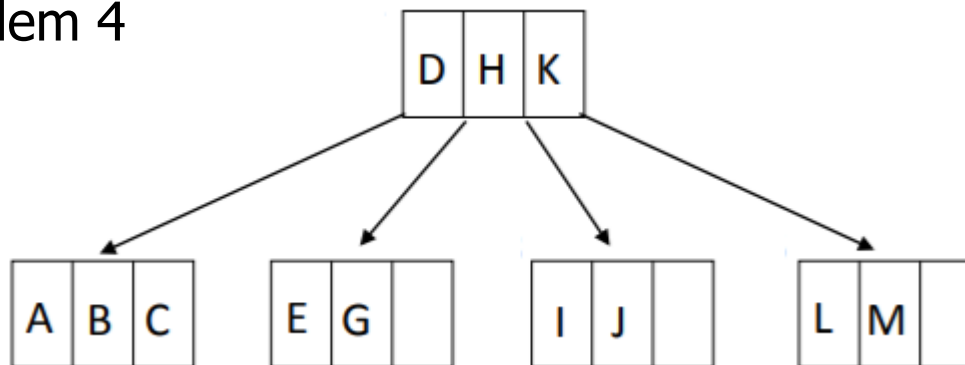
- A relação entre o vetor **chaves** e o vetor **filhos** é a seguinte:



- Note que:

- ✓ **filhos[i]** contém as chaves **menores** que **chaves[i]**, para $i < \text{num_chaves}$
- ✓ **filhos[num_chaves]** contém as chaves **maiores** que **chaves[num_chaves-1]**

➤ Exemplo: ordem 4



```
typedef struct no_btree NoBTree;

// Cria um nó vazio.
NoBTree *cria();

// Busca um valor na árvore e retorna uma referência para o nó e a
// posição do valor. Se não achar retorna NULO.
NoBTree *busca(NoBTree *r, int valor, int *posicao);

// Insere um novo nó com o valor especificado.
// Retorna true ou false, se a operação foi bem sucedida ou não.
bool insere(NoBTree *r, int valor);

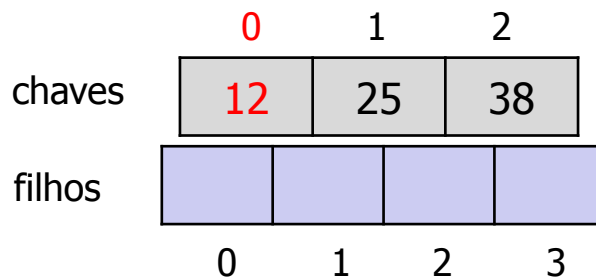
// Visita os nós da usando a estratégia em ordem.
void emOrdem(NoBTree *r);

// Exclui todos os nós de uma árvore, liberando a memória.
void libera(NoBTree *r);
```

- A busca em uma Árvore B adota a mesma estratégia das demais árvores de busca.

```
NoBTree *busca(NoBTree *r, int valor, int *posicao) {  
    if (r == NULL) {  
        *posicao = -1;  
        return NULL;  
    }  
  
    for (int i = 0; i < r->num_chaves; i++)  
        if (r->chaves[i] == valor) {  
            *posicao = i;  
            return r;  
        }  
    else if (valor < r->chaves[i])  
        return busca(r->filhos[i], valor, posicao);  
  
    return busca(r->filhos[r->num_chaves], valor, posicao);  
}
```

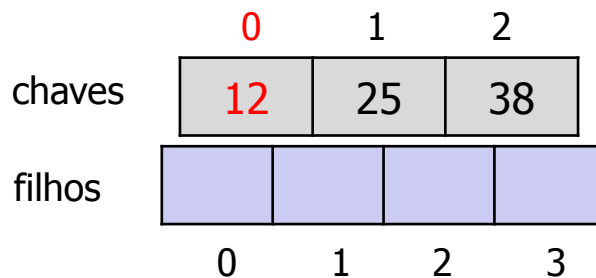
➤ Exemplo 1: busca 8.



```
for (int i = 0; i < r->num_chaves; i++)
    if (r->chaves[i] == valor) {
        *posicao = i;
        return r;
    }
    else if (valor < r->chaves[i])
        return busca(r->filhos[i], valor, posicao);

return busca(r->filhos[r->num_chaves], valor, posicao);
```

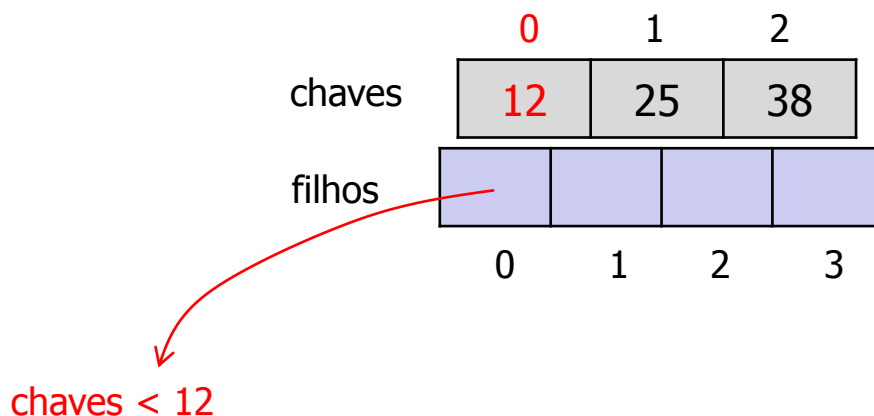
➤ Exemplo 1: busca 8.



```
for (int i = 0; i < r->num_chaves; i++)
    if (r->chaves[i] == valor) {
        *posicao = i;
        return r;
    }
    else if (valor < r->chaves[i])
        return busca(r->filhos[i], valor, posicao);

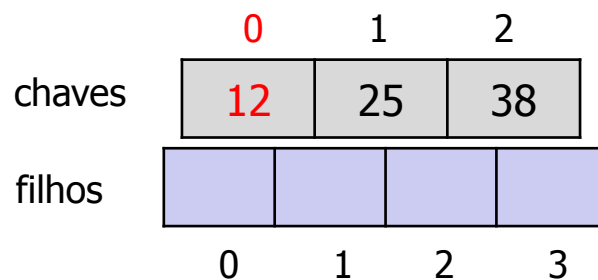
return busca(r->filhos[r->num_chaves], valor, posicao);
```

➤ Exemplo 1: busca 8.



```
for (int i = 0; i < r->num_chaves; i++)  
    if (r->chaves[i] == valor) {  
        *posicao = i;  
        return r;  
    }  
    else if (valor < r->chaves[i])  
        return busca(r->filhos[i], valor, posicao);  
  
return busca(r->filhos[r->num_chaves], valor, posicao);
```

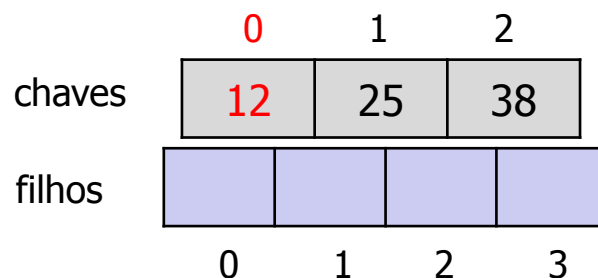
➤ Exemplo 2: busca 17.



```
for (int i = 0; i < r->num_chaves; i++)
    if (r->chaves[i] == valor) {
        *posicao = i;
        return r;
    }
else if (valor < r->chaves[i])
    return busca(r->filhos[i], valor, posicao);

return busca(r->filhos[r->num_chaves], valor, posicao);
```

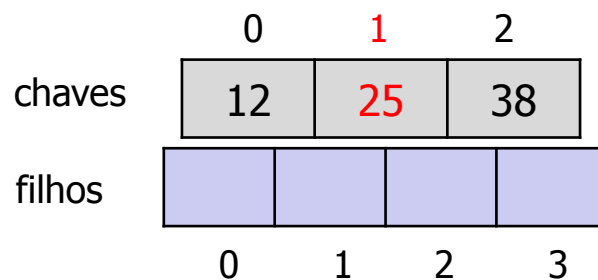
➤ Exemplo 2: busca 17.



```
for (int i = 0; i < r->num_chaves; i++)
    if (r->chaves[i] == valor) {
        *posicao = i;
        return r;
    }
    else if (valor < r->chaves[i])
        return busca(r->filhos[i], valor, posicao);

return busca(r->filhos[r->num_chaves], valor, posicao);
```

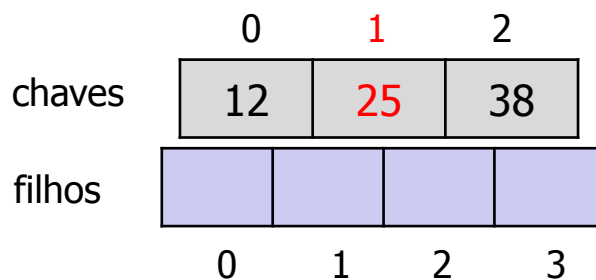
➤ Exemplo 2: busca 17.



```
for (int i = 0; i < r->num_chaves; i++)
    if (r->chaves[i] == valor) {
        *posicao = i;
        return r;
    }
else if (valor < r->chaves[i])
    return busca(r->filhos[i], valor, posicao);

return busca(r->filhos[r->num_chaves], valor, posicao);
```

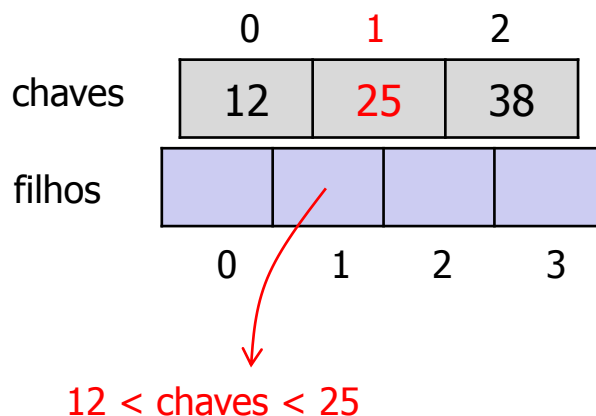
➤ Exemplo 2: busca 17.



```
for (int i = 0; i < r->num_chaves; i++)
    if (r->chaves[i] == valor) {
        *posicao = i;
        return r;
    }
    else if (valor < r->chaves[i])
        return busca(r->filhos[i], valor, posicao);

return busca(r->filhos[r->num_chaves], valor, posicao);
```

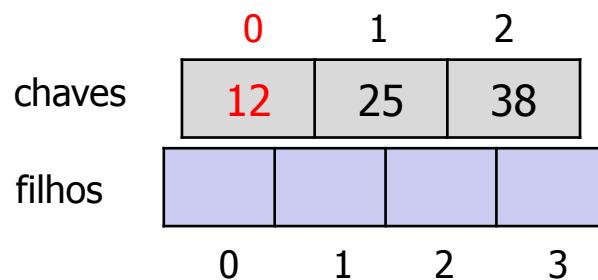
➤ Exemplo 2: busca 17.



```
for (int i = 0; i < r->num_chaves; i++)
    if (r->chaves[i] == valor) {
        *posicao = i;
        return r;
    }
    else if (valor < r->chaves[i])
        return busca(r->filhos[i], valor, posicao);

return busca(r->filhos[r->num_chaves], valor, posicao);
```

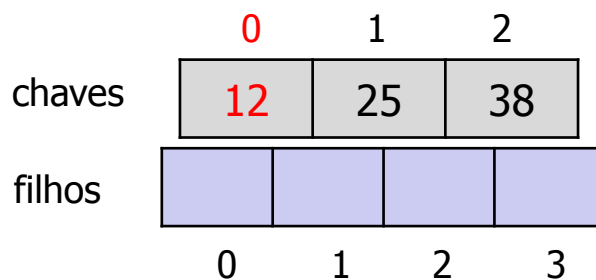
➤ Exemplo 3: busca 42.



```
for (int i = 0; i < r->num_chaves; i++)
    if (r->chaves[i] == valor) {
        *posicao = i;
        return r;
    }
else if (valor < r->chaves[i])
    return busca(r->filhos[i], valor, posicao);

return busca(r->filhos[r->num_chaves], valor, posicao);
```

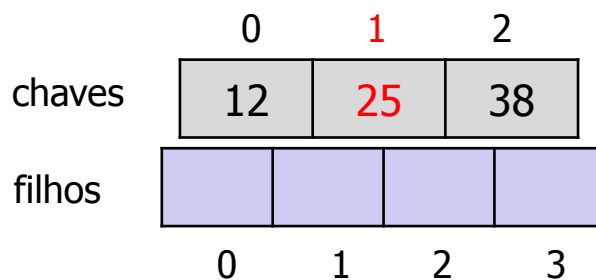
➤ Exemplo 3: busca 42.



```
for (int i = 0; i < r->num_chaves; i++)
    if (r->chaves[i] == valor) {
        *posicao = i;
        return r;
    }
    else if (valor < r->chaves[i])
        return busca(r->filhos[i], valor, posicao);

return busca(r->filhos[r->num_chaves], valor, posicao);
```

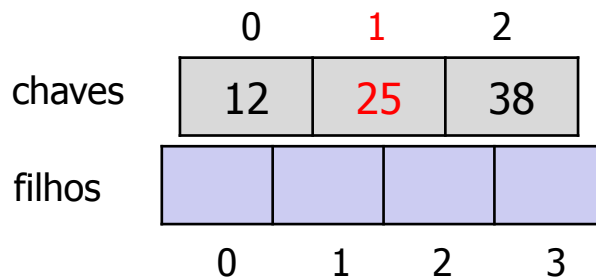
➤ Exemplo 3: busca 42.



```
for (int i = 0; i < r->num_chaves; i++)
    if (r->chaves[i] == valor) {
        *posicao = i;
        return r;
    }
    else if (valor < r->chaves[i])
        return busca(r->filhos[i], valor, posicao);

return busca(r->filhos[r->num_chaves], valor, posicao);
```

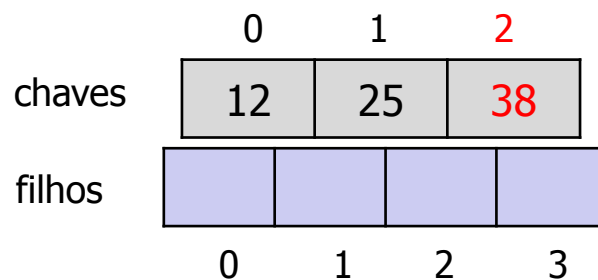
➤ Exemplo 3: busca 42.



```
for (int i = 0; i < r->num_chaves; i++)
    if (r->chaves[i] == valor) {
        *posicao = i;
        return r;
    }
    else if (valor < r->chaves[i])
        return busca(r->filhos[i], valor, posicao);

return busca(r->filhos[r->num_chaves], valor, posicao);
```

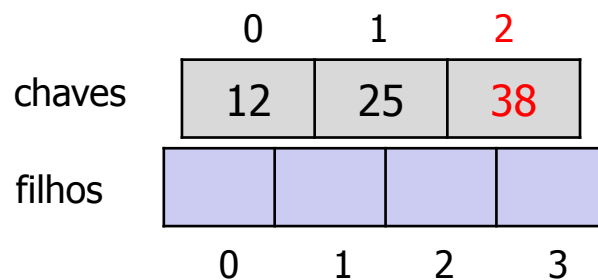
➤ Exemplo 3: busca 42.



```
for (int i = 0; i < r->num_chaves; i++)
    if (r->chaves[i] == valor) {
        *posicao = i;
        return r;
    }
    else if (valor < r->chaves[i])
        return busca(r->filhos[i], valor, posicao);

return busca(r->filhos[r->num_chaves], valor, posicao);
```

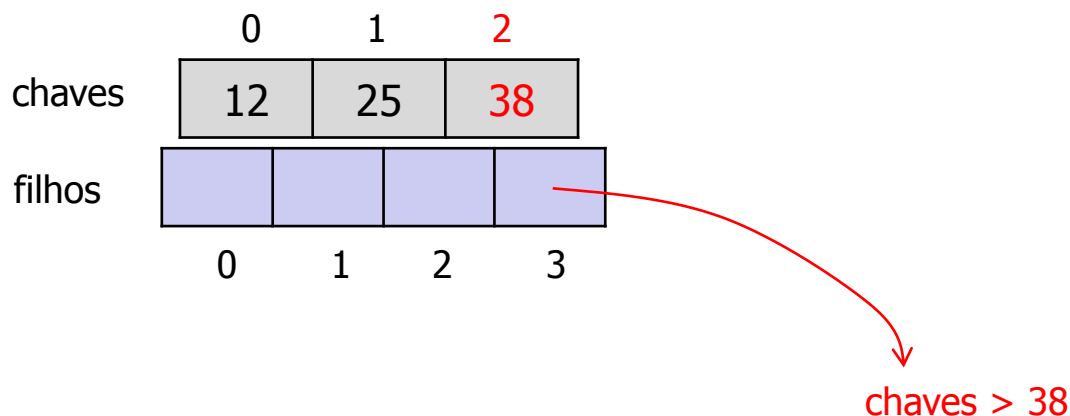
➤ Exemplo 3: busca 42.



```
for (int i = 0; i < r->num_chaves; i++)
    if (r->chaves[i] == valor) {
        *posicao = i;
        return r;
    }
    else if (valor < r->chaves[i])
        return busca(r->filhos[i], valor, posicao);

return busca(r->filhos[r->num_chaves], valor, posicao);
```

➤ Exemplo 3: busca 42.

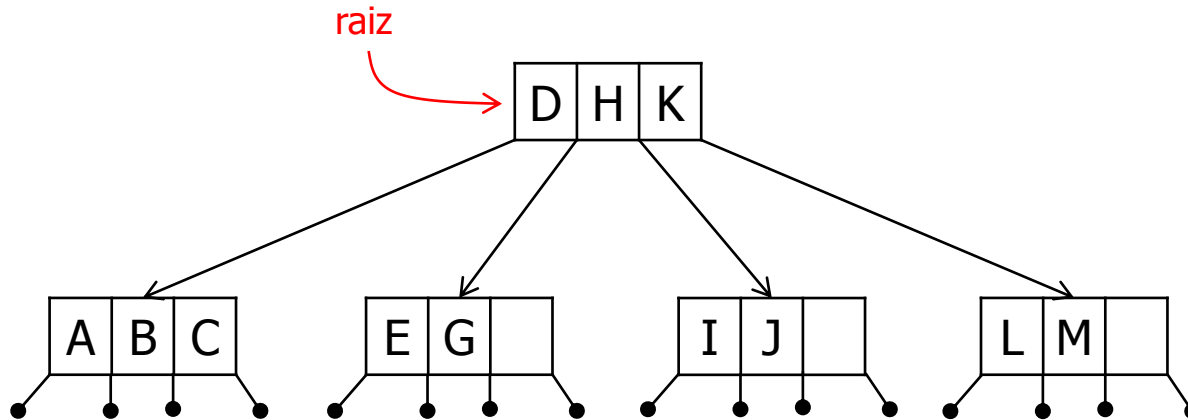


```
for (int i = 0; i < r->num_chaves; i++)  
    if (r->chaves[i] == valor) {  
        *posicao = i;  
        return r;  
    }  
    else if (valor < r->chaves[i])  
        return busca(r->filhos[i], valor, posicao);  
  
return busca(r->filhos[r->num_chaves], valor, posicao);
```

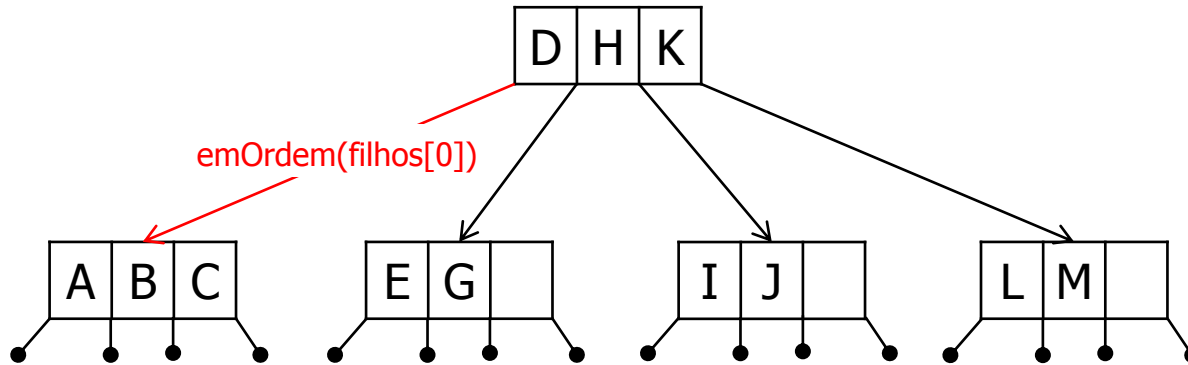
- Por ser uma Árvore de Busca, o percurso **em ordem** visita os nós da Árvore B **em ordem crescente**.
- Note que é preciso **intercalar** a visita à subárvore à esquerda com a visita dos nós. A última subárvore à direita é visitada por último.

```
void emOrdem(NoBTree *r) {  
    if (r != NULL) {  
        for (int i = 0; i < r->num_chaves; i++) {  
            if (! r->folha)  
                emOrdem(r->filhos[i]);  
            printf("%d ", r->chaves[i]);  
        }  
        if (! r->folha)  
            emOrdem(r->filhos[r->num_chaves]);  
    }  
}
```

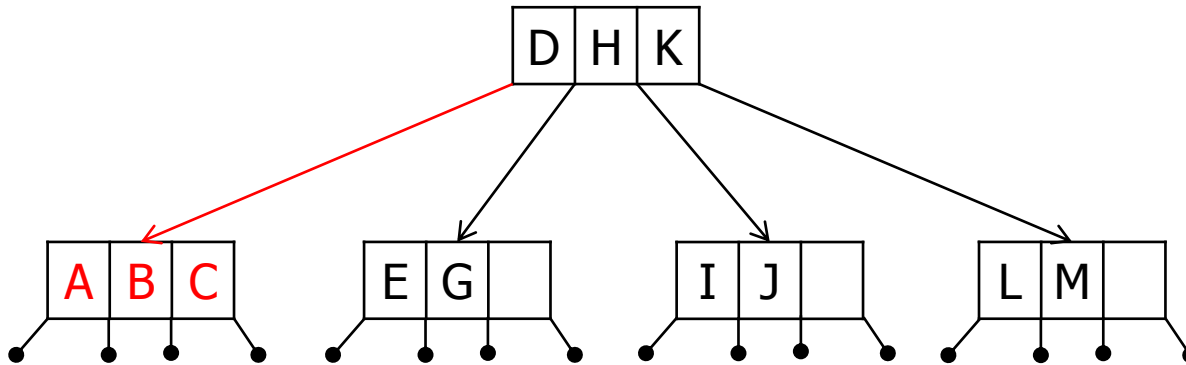
➤ Exemplo:



➤ Exemplo:

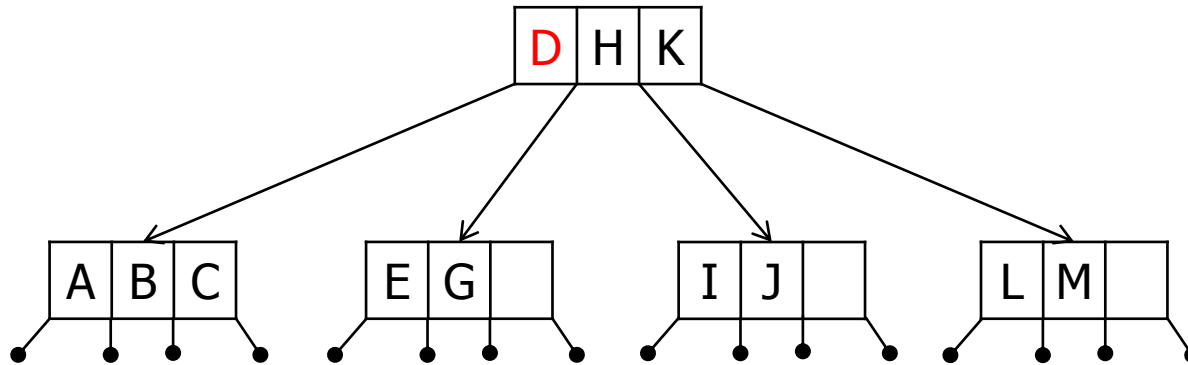


➤ Exemplo:



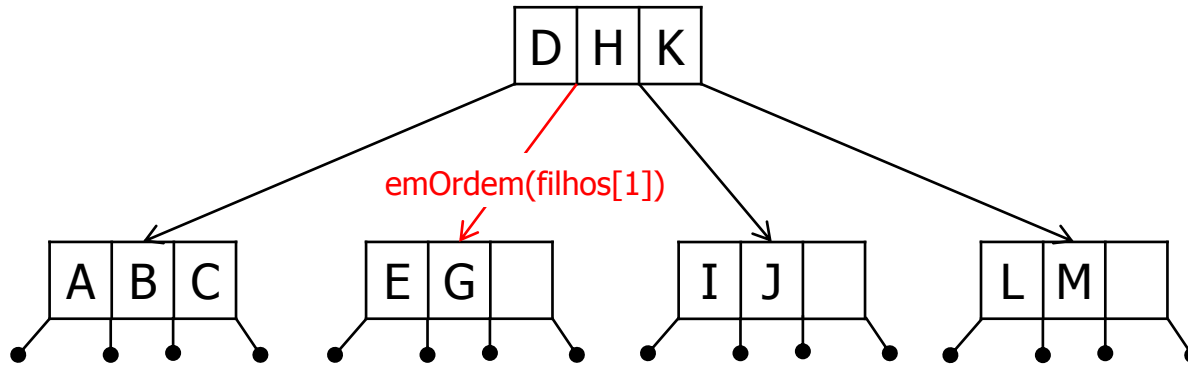
A B C

➤ Exemplo:



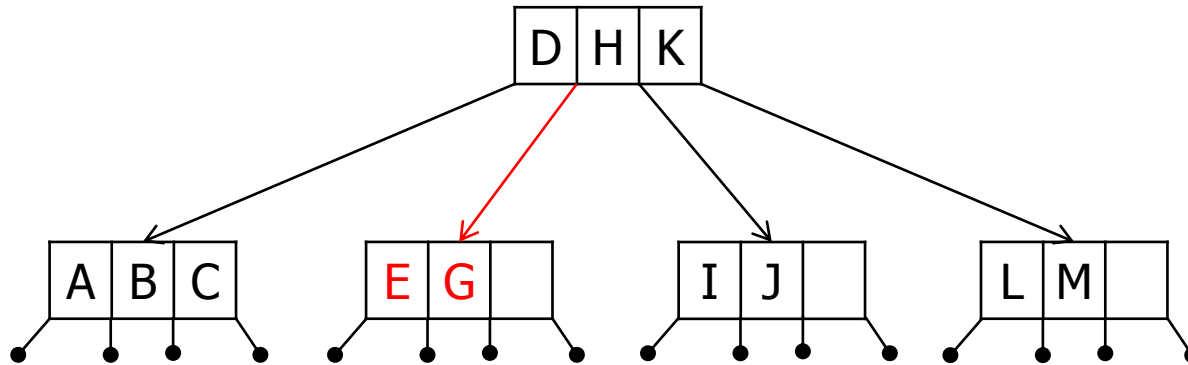
A B C D

➤ Exemplo:



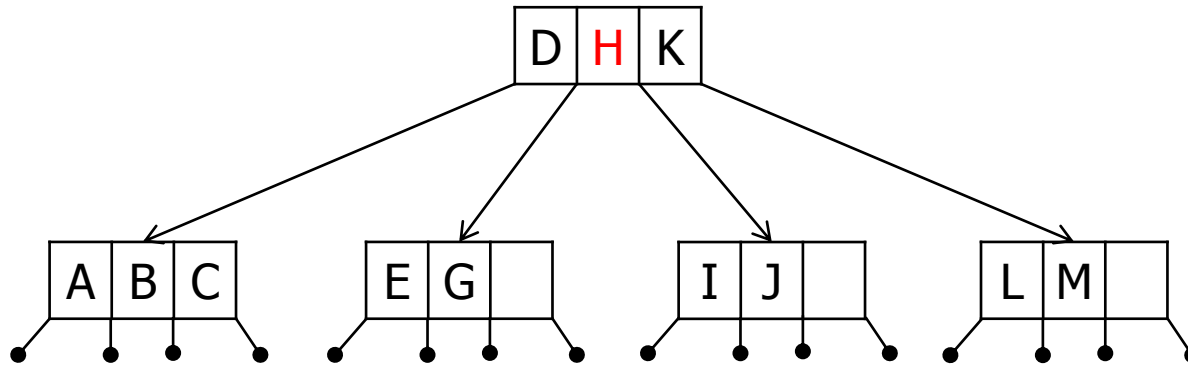
A B C D

➤ Exemplo:



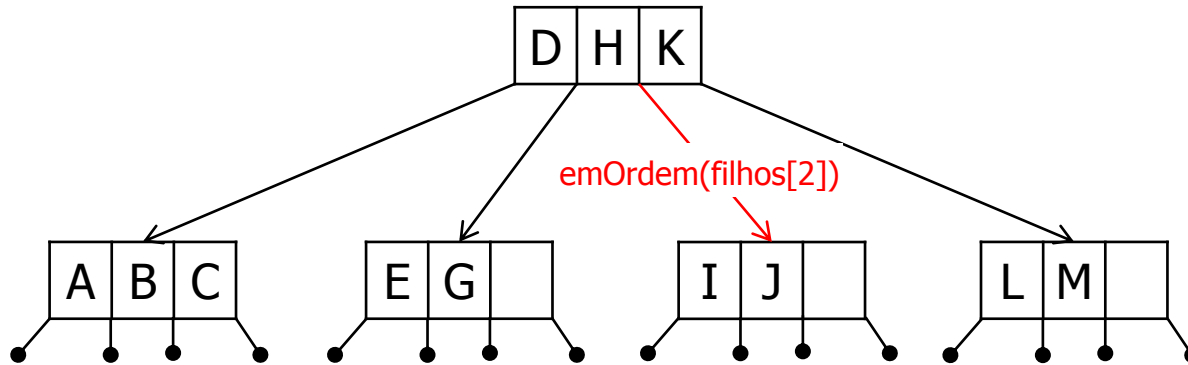
A B C D E G

➤ Exemplo:



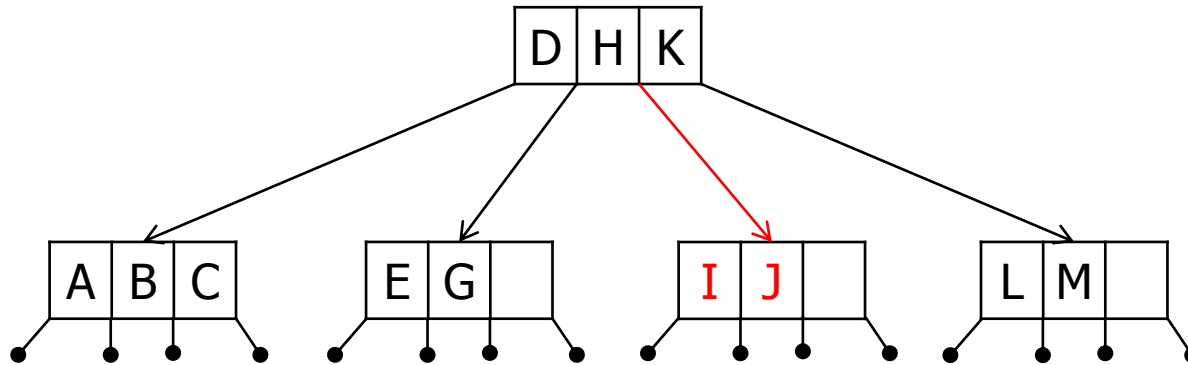
A B C D E G H

➤ Exemplo:



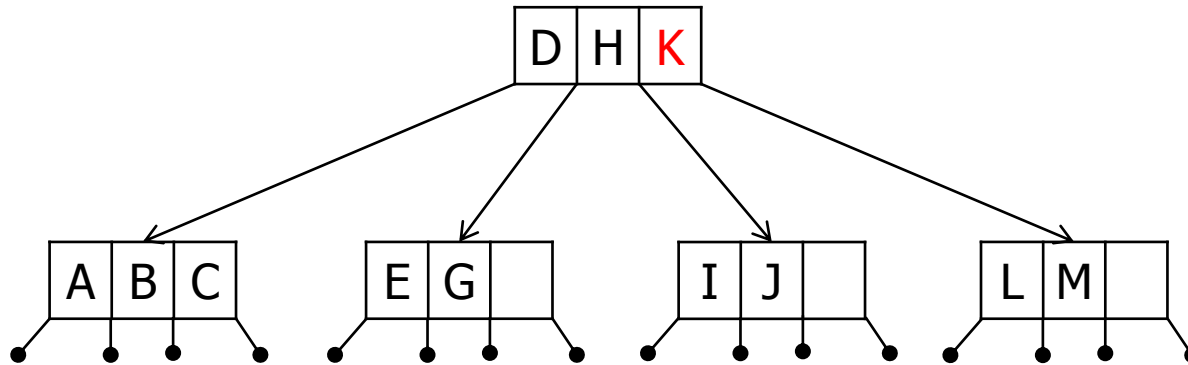
A B C D E G H

➤ Exemplo:



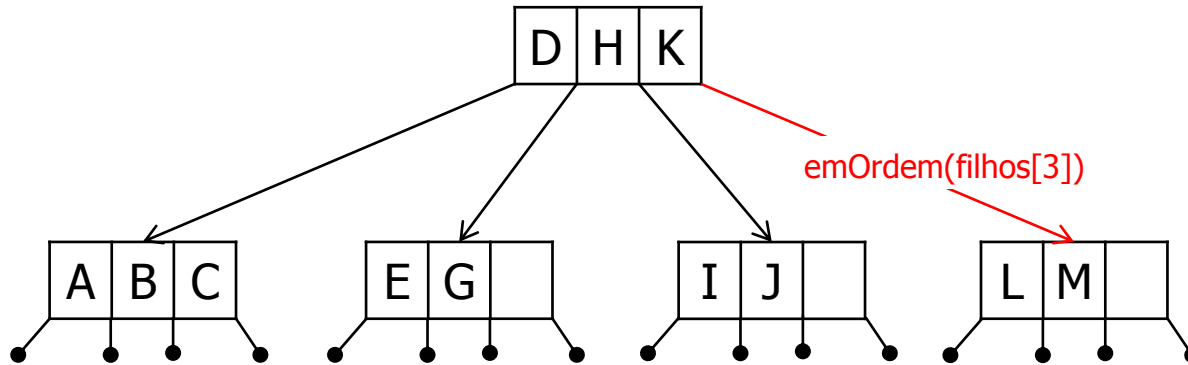
A B C D E G H I J

➤ Exemplo:



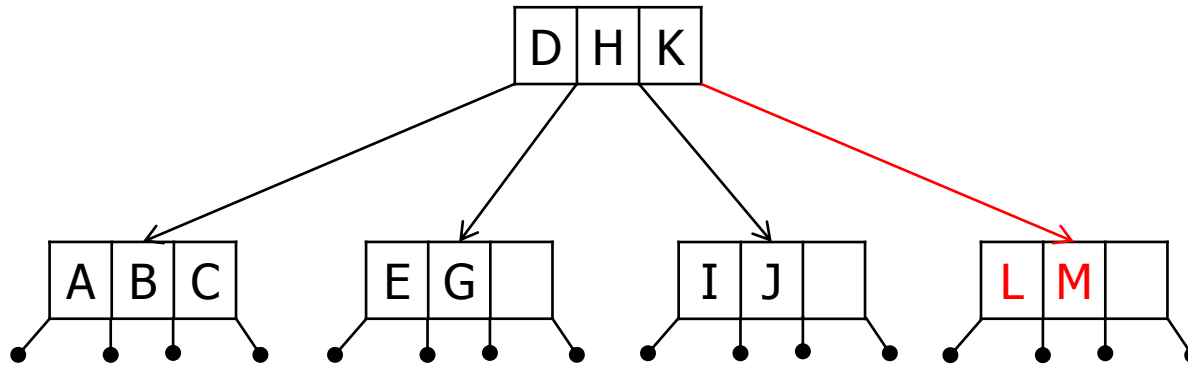
A B C D E G H I J K

➤ Exemplo:



A B C D E G H I J K

➤ Exemplo:



A B C D E G H I J K L M

- O algoritmo de inserção em Árvores B é complexo porque a árvore resultante tem que manter as propriedades apresentadas anteriormente.
- Principal característica:
 - ✓ A inserção ocorre **sempre e somente em nós folha**.
- Situações a serem analisadas:
 1. Árvore vazia
 2. Overflow no nó raiz
 3. Inserção nos nós folhas

➤ Ideia Básica:

1. Inserção da primeira chave:

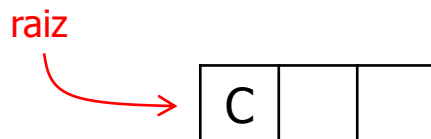
- ✓ Criação do nó raiz e inserção da chave.

2. Inserção das demais chaves:

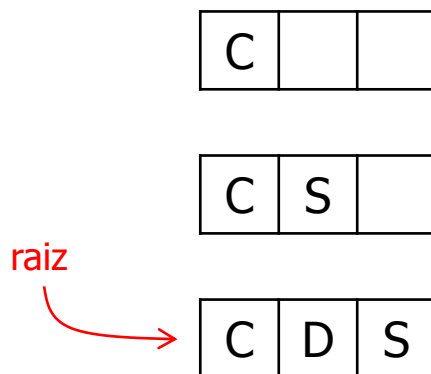
- ✓ Buscar o nó folha onde a chave deve ser inserida.
- ✓ Inserir a chave de forma ordenada no nó folha até atingir sua capacidade máxima.

➤ Vamos ver vários exemplos com uma Árvore B de **ordem 8**.

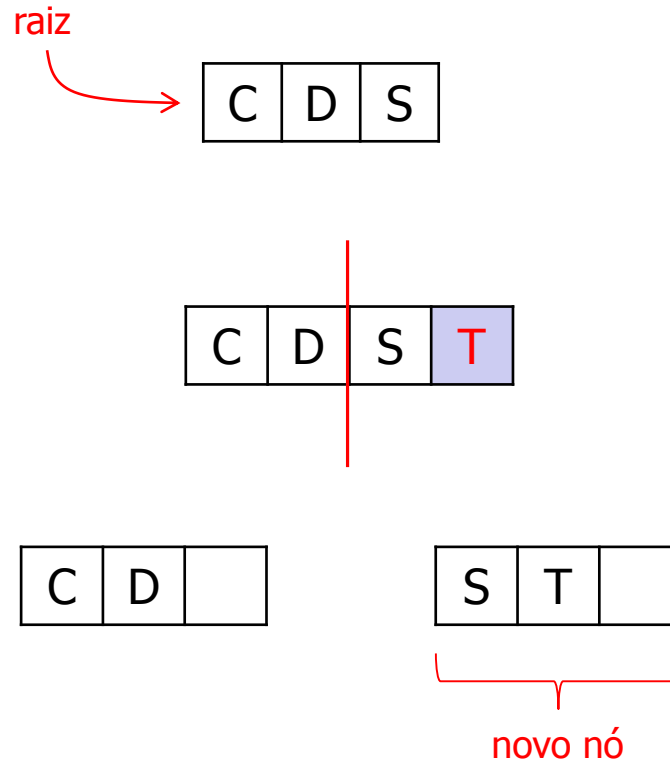
- **Árvore vazia:** inserção das chaves C S D
- Passo 1: criação do nó raiz e inserção da primeira chave.



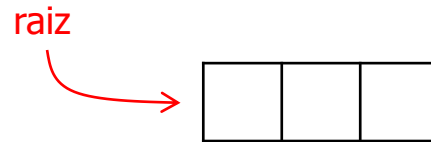
- **Árvore vazia:** inserção das chaves C S D
- Passo 2: para as demais chaves, inserir até o limite do nó.



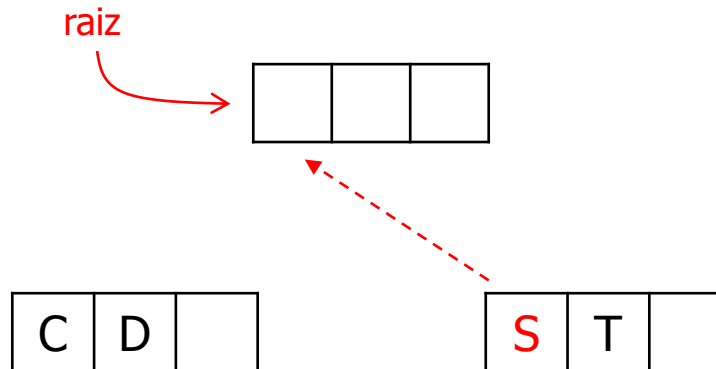
- **Overflow no nó raiz:** inserção da chave T
- **Passo 1:** se o nó está cheio (overflow), particiona o nó resultante ao meio, gerando dois nós (split).



- Overflow no nó raiz: inserção da chave T
- Passo 2: criação de uma nova raiz.

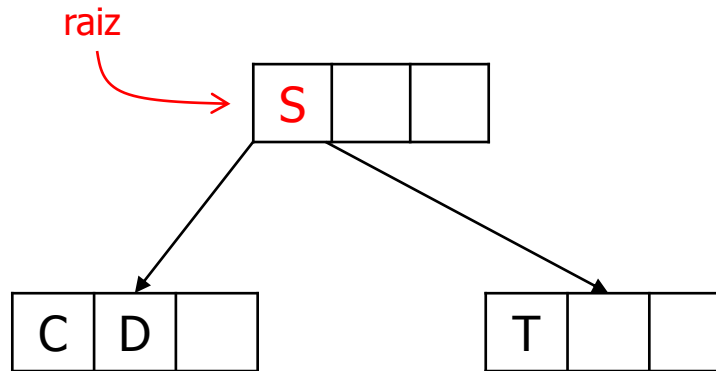


- Overflow no nó raiz: inserção da chave T
- Passo 2: promove o primeiro elemento do novo nó* para o nó raiz (promotion).

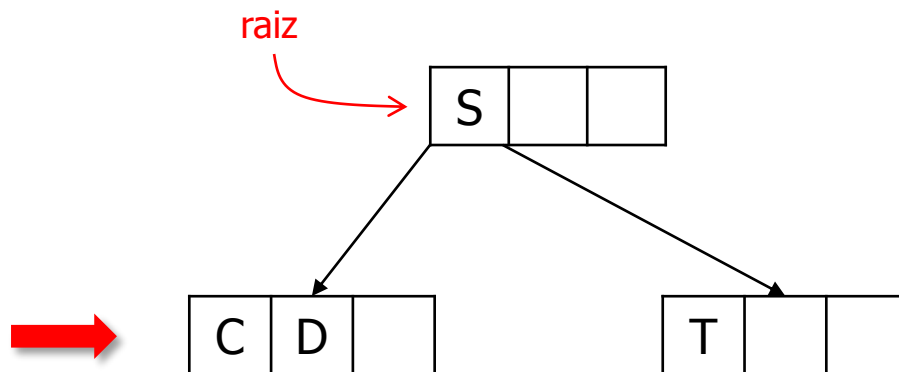


(*) outra estratégia possível é promover o último elemento do primeiro nó (nesse exemplo seria D).

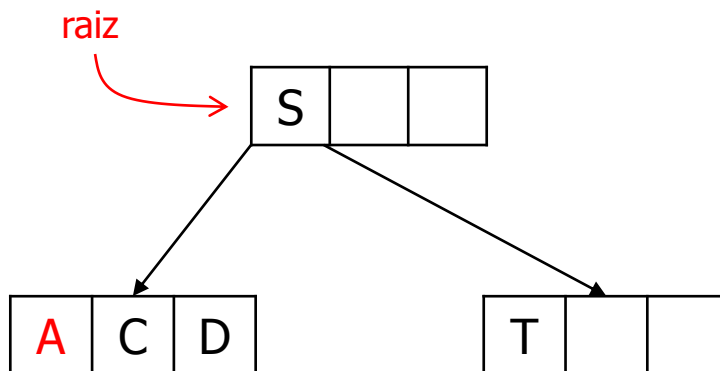
- **Overflow no nó raiz:** inserção da chave T
- **Passo 3:** ajusta os filhos esquerdo e direito da chave promovida.



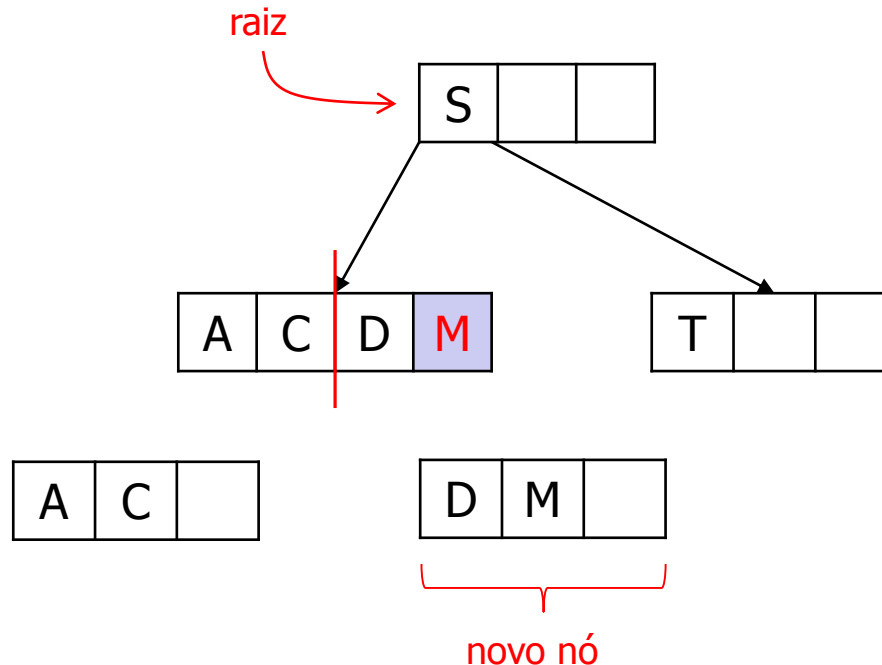
- **Inserção no nó folha:** inserção da chave A
- **Passo 1:** buscar o nó folha no qual a chave deve ser inserida.



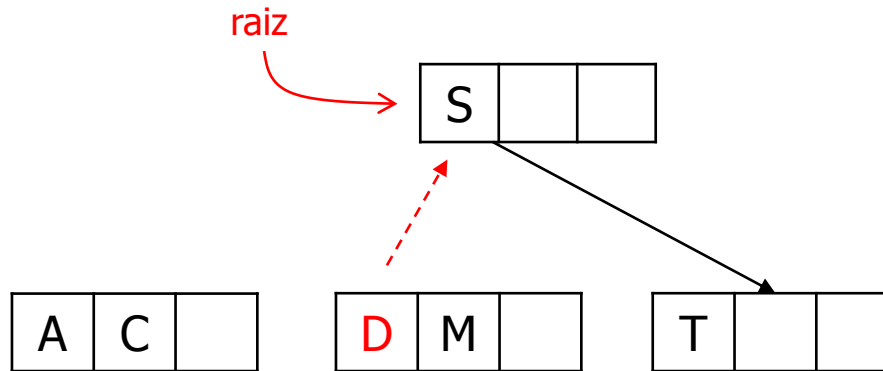
- **Inserção no nó folha:** inserção da chave A
- **Passo 2:** se houver espaço, inserir a chave de forma ordenada.



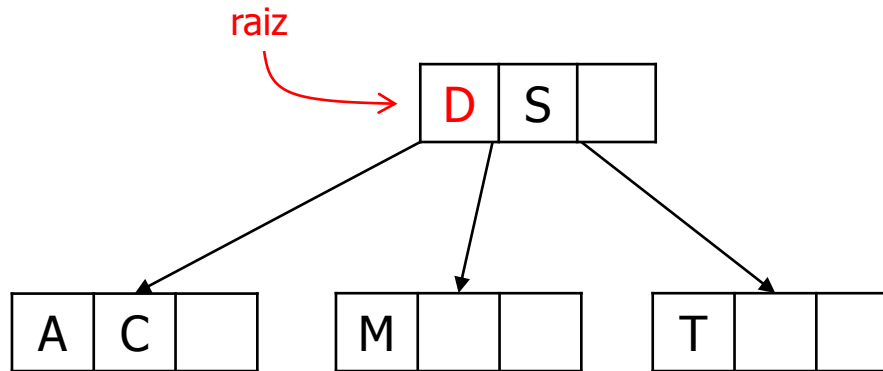
- **Inserção no nó folha:** inserção da chave M
- **Passo 2:** se nó está cheio (overflow), então particiona o nó resultante ao meio, gerando dois nós.



- **Inserção no nó folha:** inserção da chave M
- Passo 3: promove o primeiro elemento do novo nó para o nó pai (promotion).

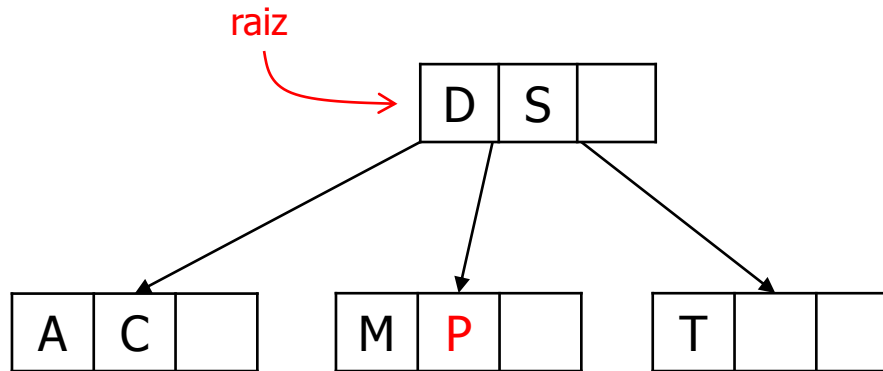


- **Inserção no nó folha:** inserção da chave M
- **Passo 4:** ajusta os filhos esquerdo e direito da chave promovida.

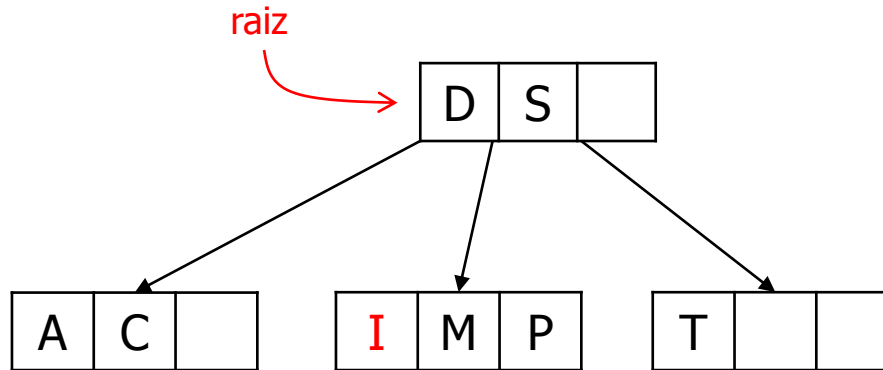


- **Observação:** caso o nó pai também esteja cheio, então o particionamento (split) e a promoção deverão ser aplicados também no nó pai, e assim, sucessivamente.

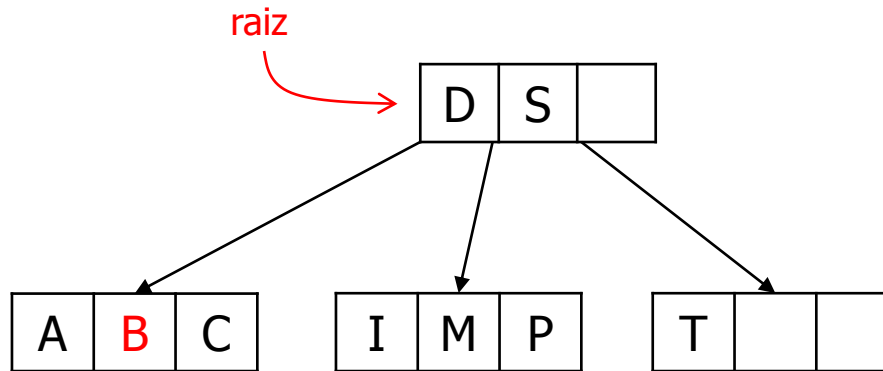
- Inserção no nó folha: inserção da chave P



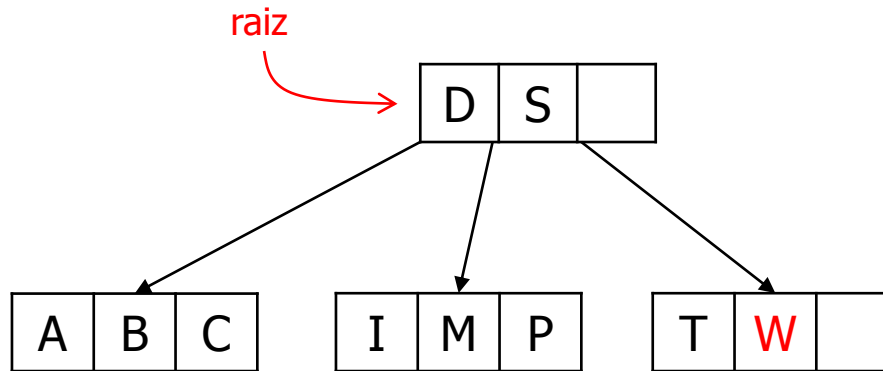
- Inserção no nó folha: inserção da chave I



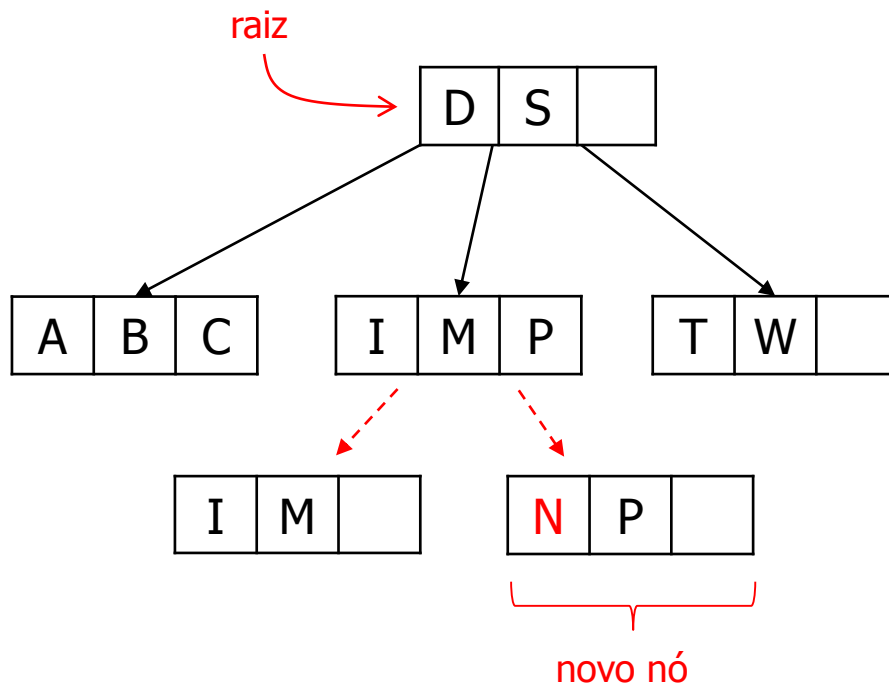
- Inserção no nó folha: inserção da chave B



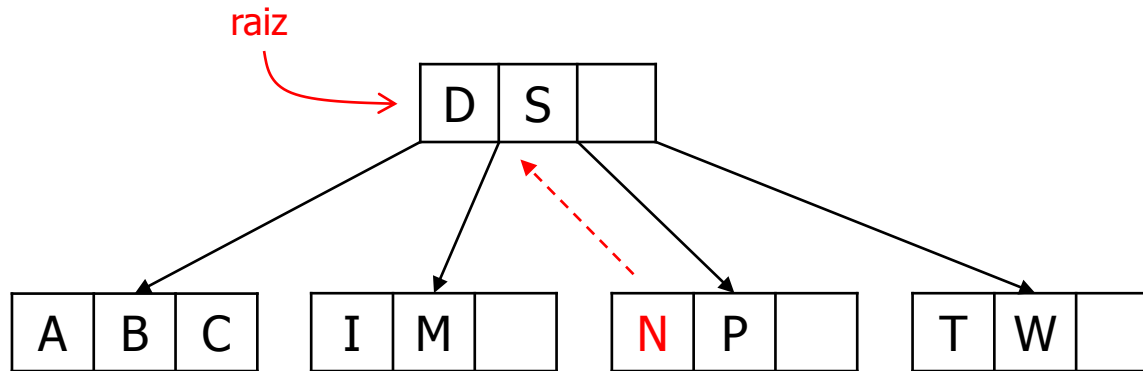
- Inserção no nó folha: inserção da chave W



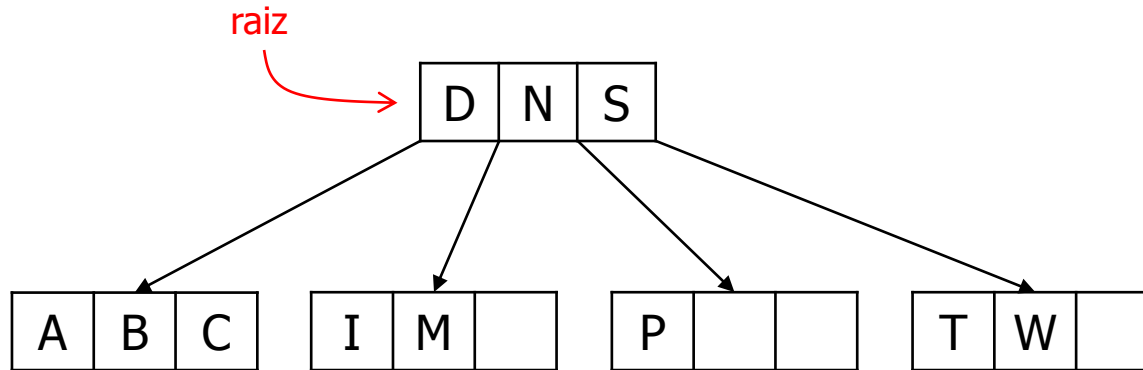
- **Inserção no nó folha:** inserção da chave N
- **Overflow no nó folha:** particiona o nó ao meio e promove o primeiro elemento do novo nó ao pai.



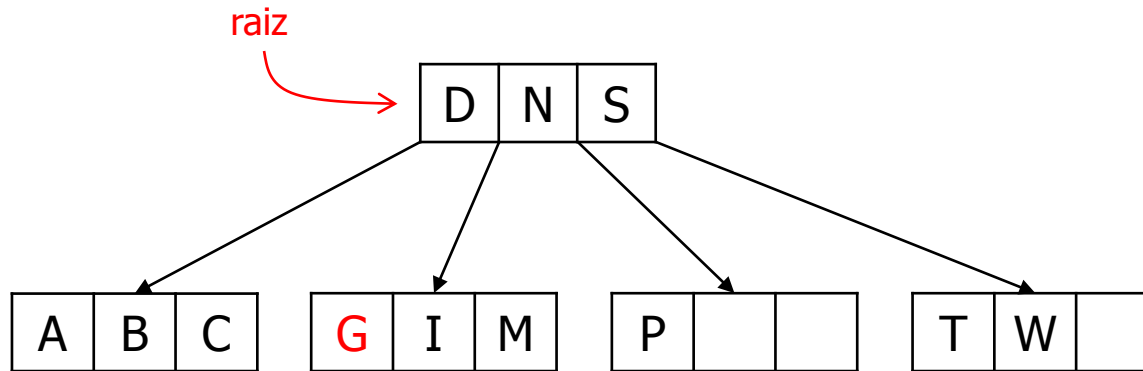
- Inserção no nó folha: inserção da chave N



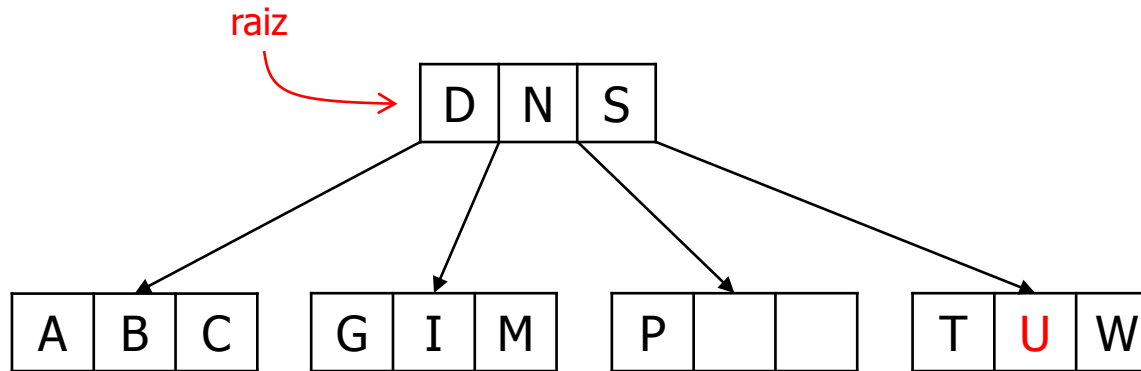
- Inserção no nó folha: inserção da chave N



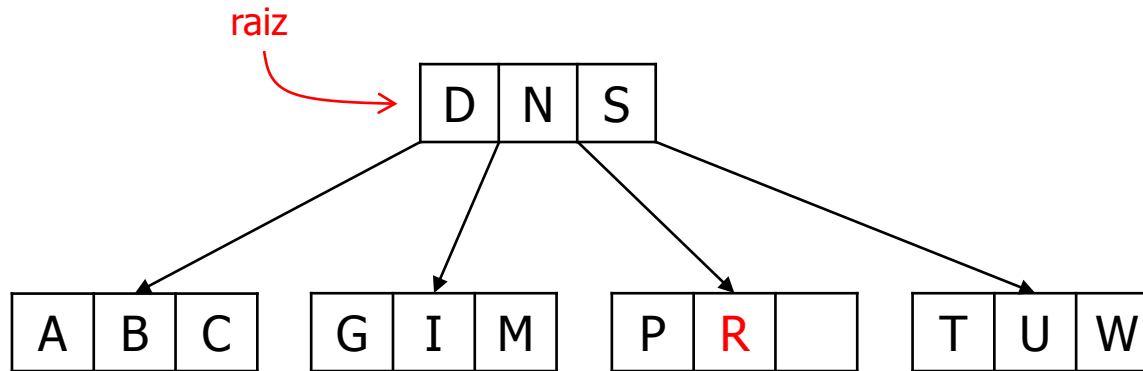
- Inserção no nó folha: inserção da chave G



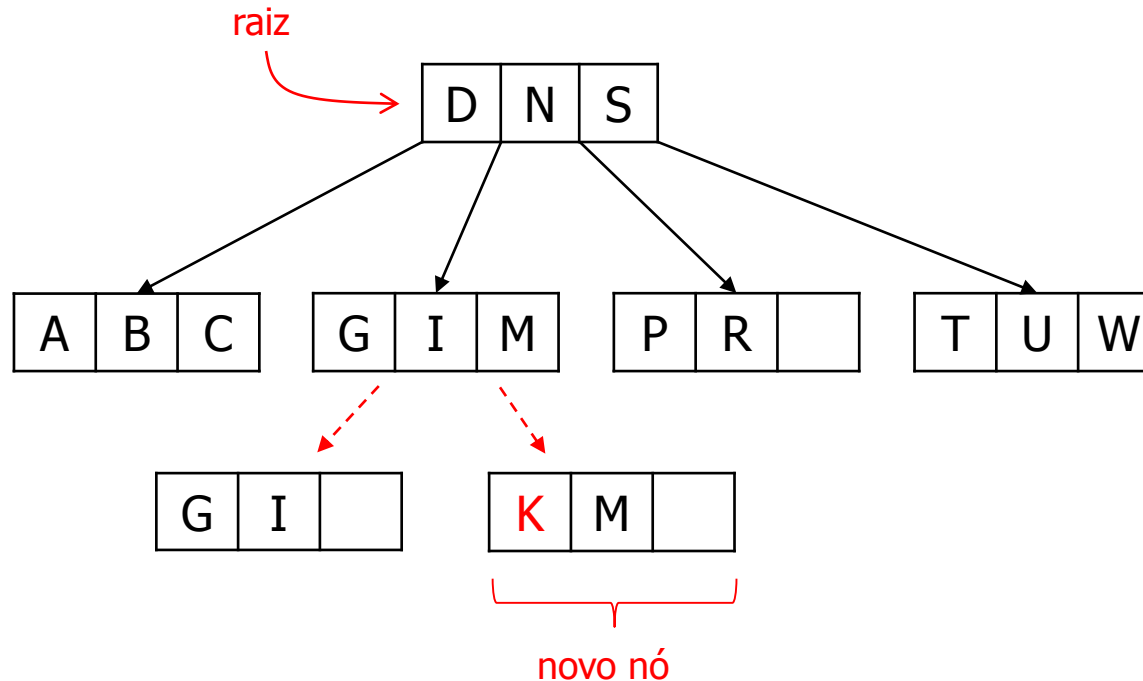
- Inserção no nó folha: inserção da chave U



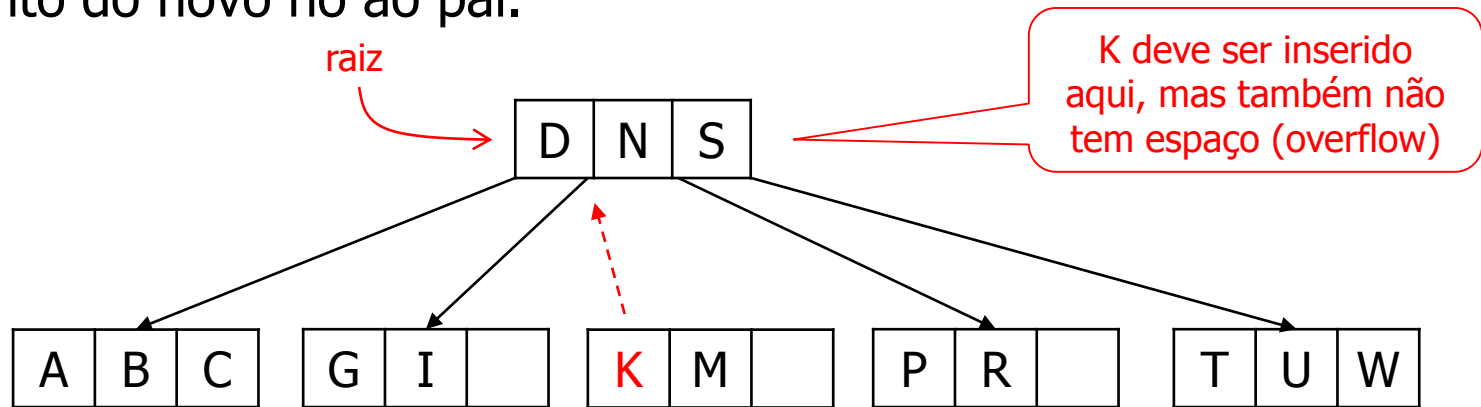
- Inserção no nó folha: inserção da chave R



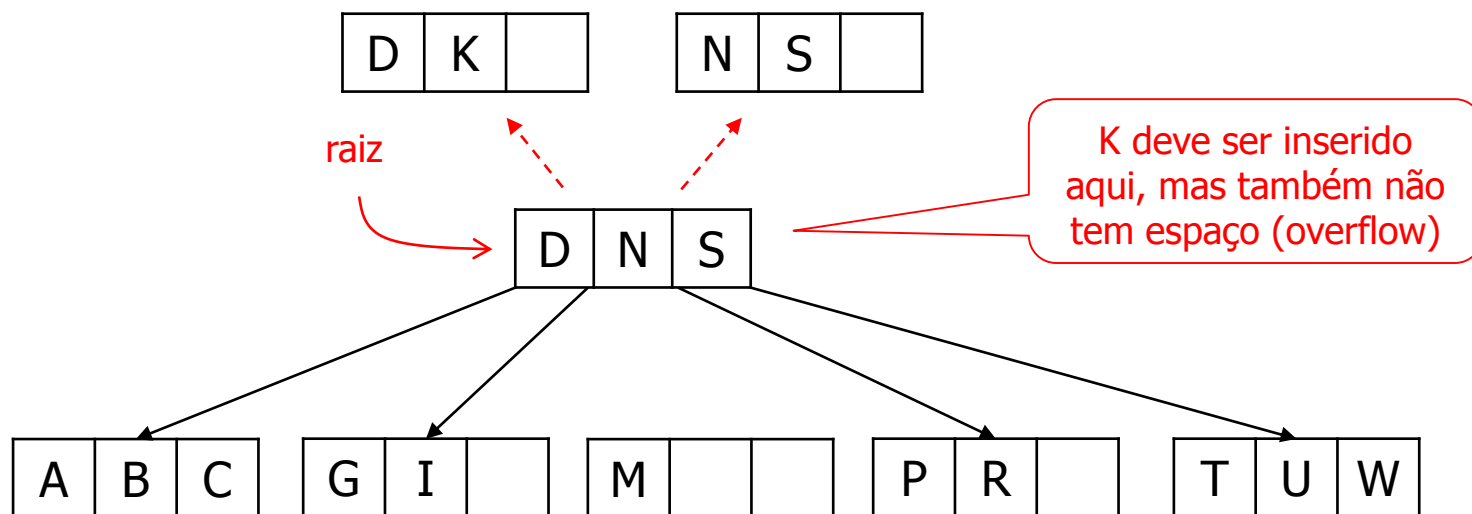
- **Inserção no nó folha:** inserção da chave K
- **Overflow no nó folha:** particiona o nó ao meio e promove o primeiro elemento do novo nó ao pai.



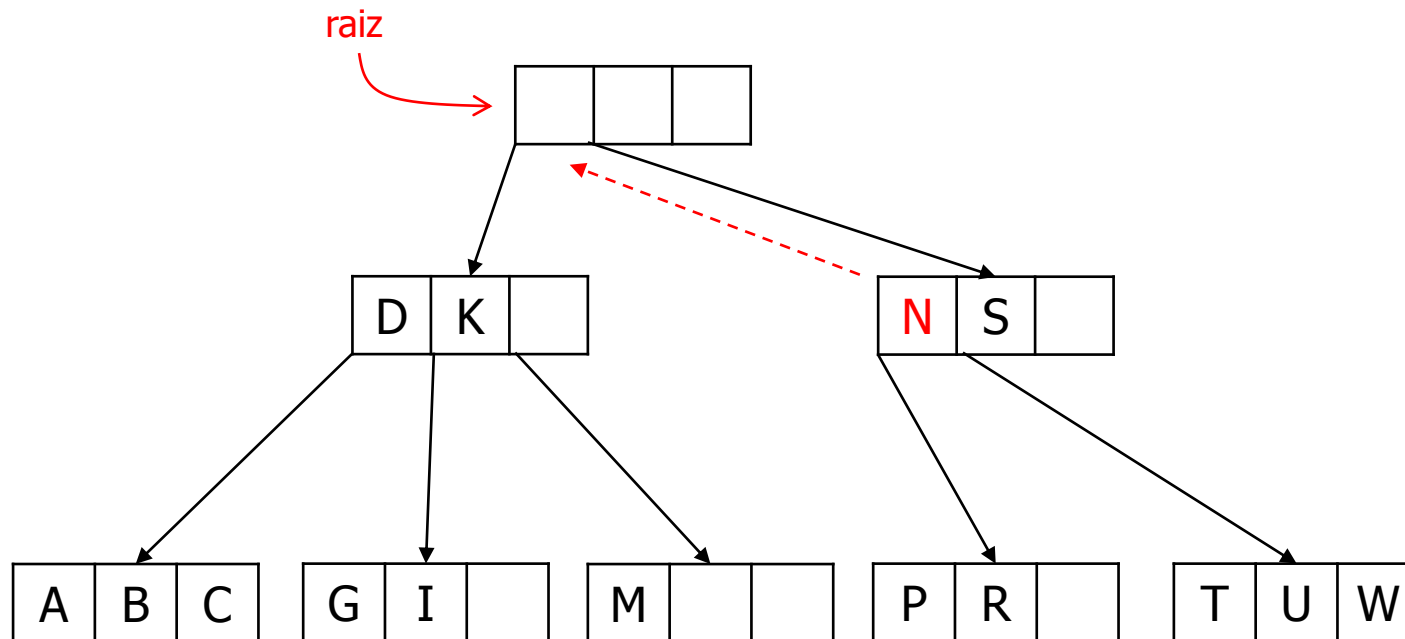
- **Inserção no nó folha:** inserção da chave K
- **Overflow no nó folha:** particiona o nó ao meio e promove o primeiro elemento do novo nó ao pai.



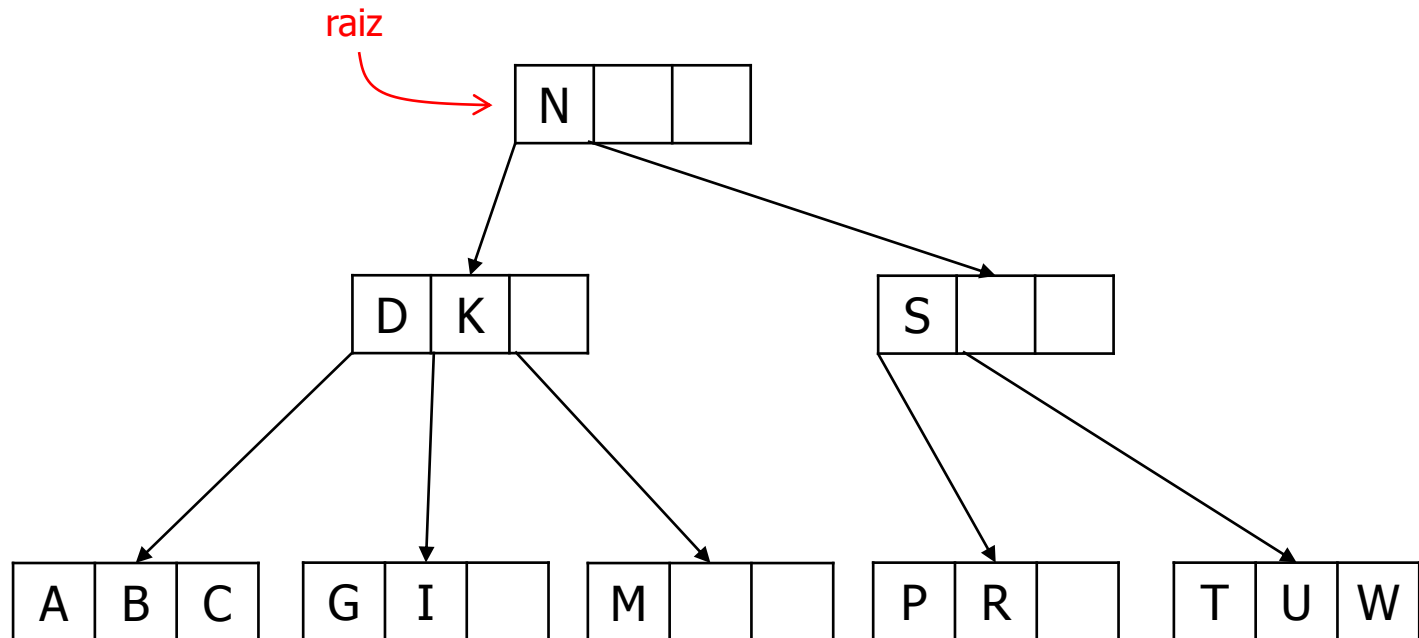
- **Inserção no nó folha:** inserção da chave K
- **Overflow na raiz:** particiona o nó ao meio, cria uma nova raiz e promove o primeiro elemento do novo nó para a raiz.



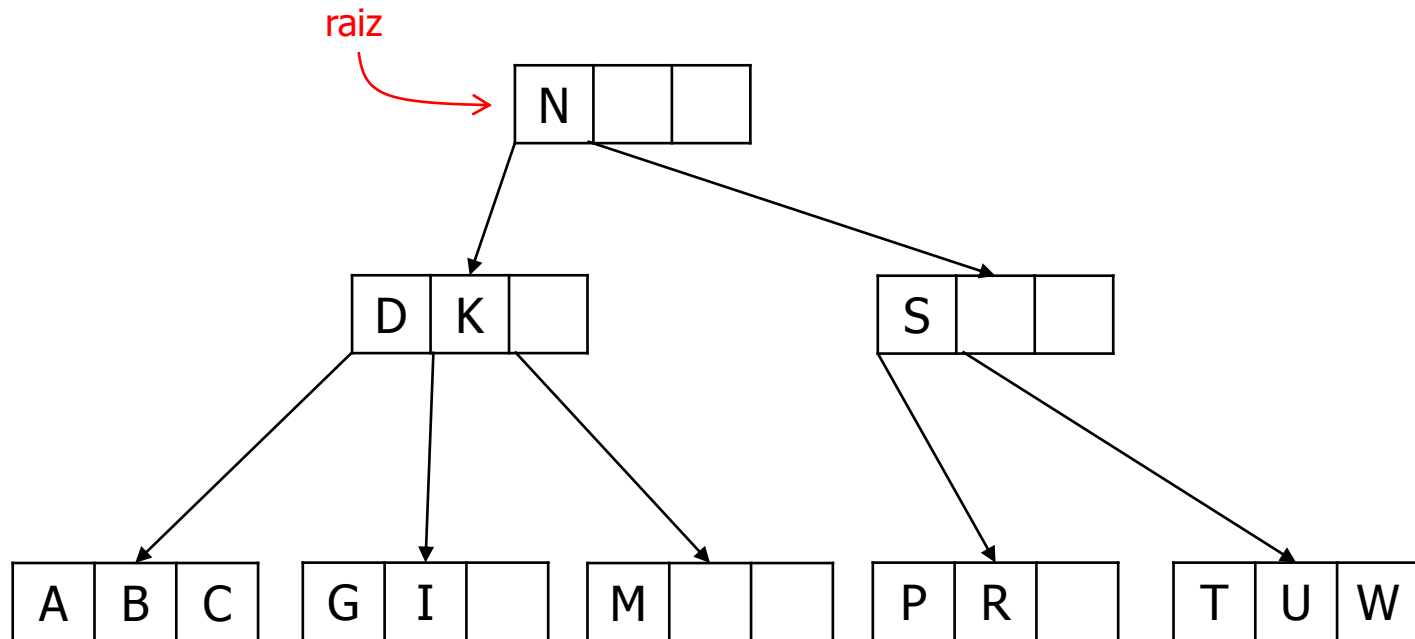
- **Inserção no nó folha:** inserção da chave K
- **Overflow na raiz:** particiona o nó ao meio, cria uma nova raiz e promove o primeiro elemento do novo nó para a raiz.



- **Inserção no nó folha:** inserção da chave K
- **Overflow na raiz:** particiona o nó ao meio, cria uma nova raiz e promove o primeiro elemento do novo nó para a raiz.



- Exercício 1: finalizar a construção da Árvore B com a inserção das chaves E H O L J Y Q Z F X V.



- Exercício 2: construa a Árvore B correspondente à inserção da sequência:

C S D T A M P I B W N G U R K E H O L J Y Q Z F X V

Durante o particionamento (split) promova o **último elemento do primeiro nó** para o nó pai.